NumPy & Pandas

Comprehensive Notes for AI & GenAI

Covers Basics → Advanced → ML/DL/GenAI Applications

Prepared: April 2025 | AI/ML Reference Guide

PART 1: NumPy — Numerical Python

1.1 What is NumPy?

NumPy (Numerical Python) is the foundational library for scientific computing in Python. It provides high-performance multi-dimensional array objects and tools for working with them. Almost every major AI/ML library — TensorFlow, PyTorch, Scikit-learn — is built on top of NumPy.

> ʿ **Key Fact:** NumPy arrays are stored in contiguous memory blocks (unlike Python lists), making them 50–100x faster for numerical operations — critical for training large AI models.

1.2 Core Data Structure: ndarray

The core object in NumPy is the ndarray (N-dimensional array). It is a grid of values of the same data type, indexed by a tuple of non-negative integers.

Key ndarray Attributes

Attribute	Description	AI/ML Relevance
ndim	Number of dimensions (axes)	Rank of tensor — 1D vector, 2D matrix, 3D batch
shape	Tuple of array dimensions	e.g., (batch_size, features) for ML datasets
size	Total number of elements	Memory footprint estimation
dtype	Data type of elements	float32 saves memory vs float64 in deep learning
strides	Bytes to step in each dimension	Used in memory-efficient tensor slicing
itemsize	Size of each element in bytes	Important for GPU memory allocation
T	Transposed array	Matrix operations, attention mechanism
flat	1-D iterator over array	Flattening layers in neural networks

1.3 Array Creation

```
import numpy as np

# Basic creation
a = np.array([1, 2, 3])                  # 1D array (vector)
b = np.array([[1,2],[3,4]])               # 2D array (matrix)
c = np.zeros((3, 4))                     # zeros — used for weight init
d = np.ones((2, 3))                      # ones  — bias initialization
e = np.eye(4)                            # identity matrix
f = np.random.randn(100, 784)            # random normal — weight init
g = np.arange(0, 10, 0.5)               # evenly spaced values
h = np.linspace(0, 1, 100)              # 100 points from 0 to 1
i = np.full((3,3), 7)                   # fill with value
j = np.empty((5, 5))                    # uninitialized (fast)
```

1.4 Data Types (dtype) — Critical for AI

Choosing the right dtype is crucial in AI/ML to balance precision and memory/speed.

dtype	Bits	Range/Precision	AI/ML Use Case
float64	64	High precision	Default Python/NumPy, scientific computing
float32	32	Single precision	Standard deep learning — PyTorch/TF default
float16	16	Half precision	Mixed precision training, GPU memory savings
bfloat16	16	Brain Float	Google TPUs, large language models (LLMs)
int32	32	±2.1 billion	Integer features, indices, class labels
int8	8	±127	Model quantization (post-training)
uint8	8	0–255	Image pixel data
bool	1	True/False	Masks, attention masks in transformers

1.5 Array Indexing & Slicing

```
a = np.array([10, 20, 30, 40, 50])
a[0]          # 10  — first element
a[-1]         # 50  — last element
a[1:4]        # [20, 30, 40]
a[::2]        # [10, 30, 50] — every 2nd

m = np.random.rand(4, 4)    # 4x4 matrix
m[0, :]       # first row
m[:, 1]       # second column
```

```
m[1:3, 1:3]   # 2x2 sub-matrix

# Boolean / fancy indexing — used for masking in transformers
mask = a > 25
a[mask]       # [30, 40, 50]
a[np.array([0, 2, 4])]      # [10, 30, 50]
```

1.6 Broadcasting — Foundation of Neural Net Math

> **AI Concept:** Broadcasting allows NumPy to operate on arrays of different shapes without copying data. This is the same mechanism used in PyTorch/TensorFlow tensors and is central to batch processing in neural networks.

```
# Scalar broadcast
x = np.array([1, 2, 3])
x * 5                   # [5, 10, 15]

# Adding bias vector to batch (shape (4,3) + (3,) -> (4,3))
batch = np.random.rand(4, 3)   # 4 samples, 3 features
bias  = np.array([1, 2, 3])    # bias vector
result = batch + bias          # bias added to each row (broadcasting)

# Broadcasting rules:
# 1. Shapes are compared from trailing dimensions
# 2. Dimensions are equal, OR one of them is 1
# 3. Missing dimensions are treated as size 1
```

1.7 Universal Functions (ufuncs)

Ufuncs operate element-wise on ndarrays. They are vectorized, meaning they avoid Python for-loops and map to C-level operations — orders of magnitude faster.

Category	Functions	AI/ML Application
Math	np.sqrt, np.exp, np.log, np.abs, np.power	Activation functions, loss calculations
Trig	np.sin, np.cos, np.tan	Positional encodings in transformers
Comparison	np.maximum, np.minimum, np.clip	ReLU: np.maximum(0, x)
Statistical	np.sum, np.mean, np.std, np.var	Batch normalization, loss aggregation
Rounding	np.floor, np.ceil, np.round	Quantization, discretization
Logical	np.logical_and, np.where	Attention masks, conditional ops

1.8 Linear Algebra — Core of Deep Learning

> ‘ **Key Fact:** Every forward pass in a neural network is fundamentally a series of matrix multiplications and dot products. NumPy's linalg module provides these operations, which are accelerated by BLAS/LAPACK under the hood.

```
import numpy as np

A = np.random.rand(3, 4)    # (3x4) weight matrix
x = np.random.rand(4)       # input vector

# Dot product / Matrix-vector multiply  (linear layer: Ax)
np.dot(A, x)                # shape (3,)
A @ x                       # same, using @ operator

# Matrix multiply (batch)
B = np.random.rand(4, 5)
np.matmul(A, B)             # shape (3,5)

# Linear algebra ops
np.linalg.det(A)            # determinant
np.linalg.inv(A)            # inverse
eigvals, eigvecs = np.linalg.eig(A)   # eigendecomposition
U, S, Vt = np.linalg.svd(A)           # SVD — used in LoRA for LLMs
np.linalg.norm(x)           # L2 norm — used in regularization
np.linalg.norm(x, ord=1)    # L1 norm
```

1.9 Reshaping — Reshaping Data for Models

```
x = np.arange(24)
x.reshape(4, 6)             # 2D — (batch, features)
x.reshape(2, 3, 4)          # 3D — (batch, seq_len, features)
x.reshape(2, -1)            # -1 inferred: (2, 12)
x.flatten()                 # 1D array (copy)
x.ravel()                   # 1D array (view, no copy — faster)

# Expand/squeeze dims — critical for batch processing
a = np.array([1,2,3])       # shape (3,)
np.expand_dims(a, axis=0)   # shape (1,3) — add batch dim
np.expand_dims(a, axis=1)   # shape (3,1) — column vector
b = np.array([[[1,2,3]]]) # shape (1,1,3)
np.squeeze(b)               # shape (3,) — remove size-1 dims

# Stack/concatenate — assembling mini-batches
np.concatenate([a, a], axis=0)
np.stack([a, a], axis=0)    # new axis: shape (2,3)
np.vstack([a, a])           # vertical stack
np.hstack([a, a])           # horizontal stack
```

1.10 NumPy in AI & GenAI — Key Applications

Activation Functions (implemented with NumPy)

```
x = np.linspace(-5, 5, 100)

# ReLU (Rectified Linear Unit) — most common in deep learning
relu = np.maximum(0, x)

# Sigmoid — used in binary classification, LSTM gates
sigmoid = 1 / (1 + np.exp(-x))

# Tanh — used in RNNs, LSTMs
tanh = np.tanh(x)

# Softmax — output layer for multi-class classification
def softmax(x):
    e_x = np.exp(x - np.max(x))    # subtract max for numerical stability
    return e_x / e_x.sum()

# GELU — used in BERT, GPT (GenAI models)
gelu = 0.5 * x * (1 + np.tanh(np.sqrt(2/np.pi) * (x + 0.044715 * x**3)))
```

Tokenization & Embedding Concepts

```
# Token IDs (integer representation of text tokens)
token_ids = np.array([101, 2023, 2003, 1037, 3231, 102])  # BERT-style

# Embedding matrix lookup (simplified)
vocab_size = 30522       # BERT vocabulary size
embedding_dim = 768      # BERT hidden dimension
embedding_matrix = np.random.randn(vocab_size, embedding_dim) * 0.02

# Embed tokens by indexing
token_embeddings = embedding_matrix[token_ids]  # shape (6, 768)

# Positional encoding (sinusoidal — original transformer)
def positional_encoding(seq_len, d_model):
    positions = np.arange(seq_len)[:, np.newaxis]
    dims = np.arange(d_model)[np.newaxis, :]
    angle = positions / np.power(10000, (2 * (dims // 2)) / d_model)
    pe = np.zeros((seq_len, d_model))
    pe[:, 0::2] = np.sin(angle[:, 0::2])
    pe[:, 1::2] = np.cos(angle[:, 1::2])
    return pe
```

Cosine Similarity — Used in RAG / Vector Search

```
# Cosine similarity: used to compare embeddings in RAG pipelines
def cosine_similarity(a, b):
```

```
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Batch cosine similarity (query vs all document embeddings)
def batch_cosine_sim(query, docs):
    query_norm = query / np.linalg.norm(query)
    docs_norm = docs / np.linalg.norm(docs, axis=1, keepdims=True)
    return np.dot(docs_norm, query_norm)  # shape (n_docs,)

# Top-k retrieval (used in RAG retrieval step)
scores = batch_cosine_sim(query_emb, doc_embeddings)
top_k = np.argsort(scores)[::-1][:5]    # top 5 indices
```

Attention Mechanism (Scaled Dot-Product Attention)

```
# The mathematical heart of every Transformer (GPT, BERT, LLaMA)
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]                                # key dimension
    scores = np.matmul(Q, K.T) / np.sqrt(d_k)    # scale
    if mask is not None:
        scores = np.where(mask, scores, -1e9)     # causal mask
    weights = softmax(scores)                      # attention weights
    return np.matmul(weights, V)                  # weighted values

# Q, K, V shapes in GPT-style model:
# Q: (batch, n_heads, seq_len, d_k)
# K: (batch, n_heads, seq_len, d_k)
# V: (batch, n_heads, seq_len, d_v)
```

Ṛ GenAI Relevance: NumPy concepts directly map to PyTorch/JAX tensors. Understanding NumPy broadcasting, matrix multiplication, and reshaping is essential before working with Transformers, Diffusion Models, and LLM inference code.

1.11 NumPy Important Terms Glossary

Term	Definition	AI/GenAI Context
ndarray	N-dimensional array object	Tensors in PyTorch/TF are based on this
Broadcasting	Operating on arrays of different shapes	Batch + bias addition in neural layers
Vectorization	Replacing loops with array ops	Core to fast model forward passes
ufunc	Universal function (element-wise)	Activation functions (ReLU, sigmoid)
Axis	Dimension of operation (0=rows, 1=cols)	Batch dimension (axis=0) in deep learning
Stride	Memory step between elements	Memory layout for convolutional ops

Term	Definition	AI/GenAI Context
View vs Copy	Shared vs copied memory	Zero-copy slicing saves GPU memory
SVD	Singular Value Decomposition	LoRA: Low-Rank Adaptation in LLMs
Einsum	Einstein summation notation	Compact multi-dimensional tensor ops
Meshgrid	Coordinate matrices from 1D arrays	2D positional encoding grids
argmax/argmin	Index of max/min element	Greedy decoding in LLM text generation
clip	Clamp values to range	Gradient clipping in backpropagation
norm	Vector/matrix magnitude	Weight regularization (L1, L2)

PART 2: Pandas — Python Data Analysis Library

2.1 What is Pandas?

Pandas is an open-source Python library for data manipulation and analysis. It provides two primary data structures — Series and DataFrame — that make it easy to work with structured/tabular data. In the AI/ML pipeline, Pandas is the primary tool for data ingestion, EDA (Exploratory Data Analysis), feature engineering, and preprocessing before model training.

> ˋ **Key Fact:** In a typical ML project, 70–80% of time is spent on data preparation. Pandas is the primary tool for this phase. Every dataset going into a Scikit-learn, TensorFlow, or PyTorch model typically passes through Pandas first.

2.2 Core Data Structures

Series — 1D Labeled Array

```
import pandas as pd
import numpy as np

# Series: 1D array with labels (index)
s = pd.Series([10, 20, 30, 40], index=['a','b','c','d'])
s['b']           # 20  — label-based access
s[1]             # 20  — position-based access
s.values         # numpy array: [10, 20, 30, 40]
s.index          # Index(['a','b','c','d'])
s.dtype          # int64
s.name = 'scores'
```

DataFrame — 2D Tabular Data

```
# DataFrame: 2D table with row index + column labels
df = pd.DataFrame({
    'name':    ['Alice', 'Bob', 'Carol', 'Dave'],
    'age':     [25, 30, 35, 28],
    'salary':  [50000, 60000, 70000, 55000],
    'dept':    ['AI', 'DS', 'AI', 'ML']
})

df.shape         # (4, 4) — rows x columns
df.dtypes        # column dtypes
df.info()        # summary: index, dtypes, memory
df.describe()    # stats: count, mean, std, min, max...
df.head(2)       # first 2 rows
df.tail(2)       # last 2 rows
df.columns       # column names
```

```
df.index          # row index (RangeIndex(0,4))
df.values         # numpy array of values
```

2.3 Indexing and Selection

Method	Syntax Example	Description
[]	df['age']	Select single column → Series
[[]]	df[['age','salary']]	Select multiple columns → DataFrame
.loc[]	df.loc[0, 'name']	Label-based — rows and columns by name
.iloc[]	df.iloc[0, 1]	Integer-based — positional indexing
.at[]	df.at[0, 'age']	Fast scalar access by label
.iat[]	df.iat[0, 1]	Fast scalar access by position
Boolean	df[df['age'] > 28]	Filter rows by condition
.query()	df.query('age > 28')	SQL-style filtering

```
# loc — label based
df.loc[0:2, 'name':'age']           # rows 0-2, cols name to age
df.loc[df['dept']=='AI', 'salary'] # conditional column selection

# iloc — position based
df.iloc[0:3, 1:3]                   # rows 0-2, cols 1-2

# Multi-condition filtering
df[(df['age'] > 25) & (df['dept'] == 'AI')]
df[df['dept'].isin(['AI', 'ML'])]  # isin filter
df[~df['name'].str.startswith('A')] # NOT condition
```

2.4 Handling Missing Data — Critical for ML

Important: Missing values (NaN) are a major source of errors in ML models. Models like linear regression, neural networks, and SVMs cannot handle NaN directly. Always handle missing data before training.

```
df = pd.DataFrame({'A': [1, np.nan, 3], 'B': [4, 5, np.nan], 'C': ['x','y','z']})

# Detection
df.isnull()               # boolean DataFrame
df.isnull().sum()         # count per column
df.isnull().sum() / len(df) * 100  # missing %
```

```
df.notnull()              # inverse of isnull

# Removal
df.dropna()               # drop rows with ANY NaN
df.dropna(axis=1)         # drop columns with ANY NaN
df.dropna(thresh=2)       # keep rows with at least 2 non-NaN
df.dropna(subset=['A'])   # drop only if 'A' is NaN

# Imputation (filling)
df.fillna(0)              # fill all NaN with 0
df['A'].fillna(df['A'].mean())    # fill with mean (simple imputation)
df['A'].fillna(df['A'].median()) # fill with median (outlier-robust)
df['A'].fillna(method='ffill')    # forward fill (time-series)
df['A'].fillna(method='bfill')    # backward fill (time-series)

# For ML — use Scikit-learn's SimpleImputer for pipeline consistency
```

2.5 Data Types & Type Conversion

```
df.dtypes                         # check all dtypes

# Conversions
df['age'] = df['age'].astype('float32')     # int → float32 (ML)
df['label'] = df['label'].astype('int8')    # class labels
df['date'] = pd.to_datetime(df['date'])     # string → datetime
df['num'] = pd.to_numeric(df['num'], errors='coerce')  # coerce bad → NaN

# Categorical — memory efficient for high-cardinality features
df['dept'] = df['dept'].astype('category')
df['dept'].cat.codes          # integer codes
df['dept'].cat.categories     # category labels

# Memory optimization for large ML datasets
df.memory_usage(deep=True)    # bytes per column
```

2.6 Feature Engineering — ML & AI Pipeline

String/Text Features

```
# Text preprocessing (NLP/GenAI pipelines)
df['text'] = df['text'].str.lower()          # lowercase
df['text'] = df['text'].str.strip()          # remove whitespace
df['text'] = df['text'].str.replace(r'[^a-z ]','', regex=True)  # clean
df['word_count'] = df['text'].str.split().str.len()  # token count
df['char_count'] = df['text'].str.len()              # character count

# Label Encoding — for categorical ML features
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

```
df['dept_encoded'] = le.fit_transform(df['dept'])

# One-Hot Encoding
pd.get_dummies(df, columns=['dept'], prefix='dept', drop_first=True)
```

Numerical Features

```
# Normalization (MinMax Scaling: 0 to 1)
df['age_norm'] = (df['age'] - df['age'].min()) / (df['age'].max() -
df['age'].min())

# Standardization (Z-score: mean=0, std=1)
df['age_std'] = (df['age'] - df['age'].mean()) / df['age'].std()

# Log transform — reduce skewness in features
df['salary_log'] = np.log1p(df['salary'])   # log(1 + x) — handles 0

# Binning / Discretization
df['age_bucket'] = pd.cut(df['age'], bins=[0,25,35,50],
labels=['young','mid','senior'])
df['age_qcut'] = pd.qcut(df['age'], q=4, labels=['Q1','Q2','Q3','Q4'])

# Polynomial / Interaction features
df['age_sq'] = df['age'] ** 2
df['age_salary'] = df['age'] * df['salary']
```

2.7 GroupBy — Aggregation for EDA

```
# GroupBy + Aggregate
df.groupby('dept')['salary'].mean()          # avg salary per dept
df.groupby('dept').agg({'salary': ['mean','std','count']})
df.groupby(['dept','age'])['salary'].sum()   # multi-level groupby

# Transform — returns same-shape result (useful for feature engineering)
df['dept_mean_sal'] = df.groupby('dept')['salary'].transform('mean')
df['z_within_dept'] = (df['salary'] - df['dept_mean_sal']) / \
                      df.groupby('dept')['salary'].transform('std')

# Filter groups (e.g., keep depts with > 3 members)
df.groupby('dept').filter(lambda g: len(g) > 3)
```

2.8 Merge & Join — Combining Datasets

```
# Merge (SQL-style join)
pd.merge(df1, df2, on='id', how='inner')     # inner join
pd.merge(df1, df2, on='id', how='left')      # left join
```

```
pd.merge(df1, df2, on='id', how='outer')     # full outer join
pd.merge(df1, df2, left_on='uid', right_on='user_id')  # different key names

# Concat — stack DataFrames
pd.concat([df1, df2], axis=0, ignore_index=True)  # vertical stack
pd.concat([df1, df2], axis=1)                      # horizontal stack
```

2.9 Apply & Map — Custom Transformations

```
# apply() — most flexible, runs function on rows/columns
df['salary_k'] = df['salary'].apply(lambda x: x / 1000)  # element-wise
df.apply(lambda row: row['age'] * row['salary'], axis=1)  # row-wise

# map() — element-wise on Series (simple substitution)
dept_map = {'AI': 0, 'ML': 1, 'DS': 2}
df['dept_code'] = df['dept'].map(dept_map)

# applymap() / map() on whole DataFrame (element-wise)
df.select_dtypes('number').apply(np.sqrt)   # sqrt all numeric cols

# pipe() — chain operations cleanly
result = (df
    .pipe(lambda d: d[d['age'] > 25])
    .pipe(lambda d: d.assign(bonus = d['salary'] * 0.1))
    .pipe(lambda d: d.sort_values('bonus', ascending=False))
)
```

2.10 Time Series Support

```
# Datetime indexing — crucial for temporal ML models
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')

# Time-based selection
df['2024']             # all of 2024
df['2024-01':'2024-06']  # Jan to Jun 2024

# Resampling (aggregation over time periods)
df.resample('D').mean()    # daily mean
df.resample('W').sum()     # weekly sum
df.resample('M').last()    # month-end values

# Rolling window features (for time-series ML)
df['rolling_mean_7d'] = df['value'].rolling(window=7).mean()
df['rolling_std_30d'] = df['value'].rolling(window=30).std()
df['ewm'] = df['value'].ewm(span=7).mean()   # exponential weighted

# Lag features (for prediction tasks)
```

```
df['lag_1'] = df['value'].shift(1)
df['lag_7'] = df['value'].shift(7)
```

2.11 Pandas in AI & ML Pipeline

Complete ML Data Preparation Workflow

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load data
df = pd.read_csv('dataset.csv')

# 2. EDA
print(df.shape, df.dtypes, df.describe())
print(df.isnull().sum())

# 3. Clean
df = df.dropna(subset=['target'])              # drop if target missing
df['feature1'].fillna(df['feature1'].median(), inplace=True)

# 4. Feature engineering
df['log_amount'] = np.log1p(df['amount'])
df = pd.get_dummies(df, columns=['category'])

# 5. Prepare X, y
X = df.drop('target', axis=1).values.astype('float32')
y = df['target'].values

# 6. Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 7. Scale
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)   # use same scaler!
```

Pandas for NLP & GenAI Dataset Preparation

```
# Building datasets for LLM fine-tuning / RAG
df = pd.read_csv('qa_dataset.csv')

# Text length analysis
df['token_count'] = df['text'].str.split().str.len()
df = df[df['token_count'] <= 512]     # max context length filter

# Formatting for instruction tuning (e.g., Alpaca format)
```

```
df['prompt'] = df.apply(lambda row:
    f"### Instruction:\n{row['question']}\n",
    axis=1)

# Creating train/validation/test splits
train = df.sample(frac=0.8, random_state=42)
remaining = df.drop(train.index)
valid = remaining.sample(frac=0.5, random_state=42)
test = remaining.drop(valid.index)

# Save as JSONL (Hugging Face format)
train.to_json('train.jsonl', orient='records', lines=True)
valid.to_json('valid.jsonl', orient='records', lines=True)
```

Ṛ GenAI Use Case: Pandas is widely used to build and curate instruction tuning datasets, RLHF preference datasets, and retrieval corpora for RAG systems. Tools like Hugging Face Datasets internally use Pandas-compatible Arrow tables.

2.12 Pandas Performance Tips

Tip	Technique	Speedup
Use vectorized ops	Avoid .iterrows() — use .apply() or vectorized ops	100x+
Categorical dtype	df.astype('category') for low-cardinality string cols	3–10x memory
Read_csv dtypes	Specify dtypes in pd.read_csv(dtype=...)	50% less memory
Chunking	pd.read_csv(..., chunksize=10000) for big files	Handles OOM
query() vs boolean	df.query('col > 5') is faster for complex filters	2–5x
eval()	pd.eval('A + B') avoids intermediate arrays	2–3x
Use .values/.to_numpy()	Returns numpy array — faster than pandas methods	3–5x
Avoid inplace=True	inplace is not actually faster; creates hidden copies	No benefit

2.13 Pandas Important Terms Glossary

Term	Definition	AI/GenAI Context
DataFrame	2D labeled table with heterogeneous dtypes	Primary structure for ML datasets

Term	Definition	AI/GenAI Context
Series	1D labeled array	Single feature column or prediction output
Index	Row labels of a DataFrame	Data alignment during joins/merges
NaN	Not a Number — missing value sentinel	Must handle before ML model training
Imputation	Filling missing values with estimated values	Mean/median impute numeric features
GroupBy	Split-Apply-Combine pattern	Compute per-class statistics for EDA
Merge/Join	Combining DataFrames on common keys	Joining feature tables in feature stores
One-Hot Encoding	Binary column per category value	Categorical feature encoding for ML
Label Encoding	Integer code per category (0, 1, 2...)	Tree models, embedding lookup tables
Feature Engineering	Creating new features from raw data	Most impactful step in ML pipeline
Train/Test Split	Partitioning data for unbiased eval	80/20 or 70/10/20 splits
Data Leakage	Test data info bleeding into train	Never fit scaler/encoder on test set
Resampling	Aggregating time-series by period	Temporal features for forecasting models
Vectorized Ops	Array-level instead of element-level ops	Core performance principle
Chunking	Processing files in pieces (large data)	Handling datasets > RAM

PART 3: NumPy vs Pandas — Quick Reference

Aspect	NumPy	Pandas
Core Object	ndarray (N-dimensional)	DataFrame / Series
Data Types	Homogeneous (all same dtype)	Heterogeneous (mixed dtypes)
Indexing	Integer positional	Label + positional (.loc/.iloc)
Performance	Fastest for pure math ops	Slower but richer functionality
Missing Data	np.nan (basic)	Native NaN handling with rich API
Use in ML	Tensor ops, model math	Data prep, EDA, feature engineering
Built on	C/Fortran BLAS/LAPACK	NumPy + Cython
Memory	Compact contiguous arrays	Higher overhead (index, metadata)
Dimensions	N-dimensional (any)	Mostly 1D (Series) / 2D (DataFrame)
SQL-like ops	Not supported	groupby, merge, pivot — full SQL ops

PART 4: The Full AI/ML Stack

Understanding where NumPy and Pandas fit in the broader AI stack:

Layer	Library	Role
Data Storage	CSV, Parquet, SQL, HDF5	Raw data on disk
Data Loading & Prep	Pandas	EDA, cleaning, feature engineering
Numerical Core	NumPy	Array math, linear algebra
ML Models (Classic)	Scikit-learn	Regression, classification, clustering
Deep Learning	PyTorch / TensorFlow	Neural networks — built on NumPy concepts
GenAI / LLMs	Hugging Face Transformers	Pre-trained models (BERT, GPT, LLaMA)
Vector Search	FAISS / Chroma / Pinecone	Embedding retrieval for RAG
Visualization	Matplotlib / Seaborn	EDA plots (built on NumPy/Pandas)

PART 5: Cheat Sheet — AI/ML Critical Operations

NumPy Cheat Sheet

```
# --- ARRAY CREATION ---
np.zeros((n, d))            # zero weight initialization
np.ones((n,))               # bias initialization
np.random.randn(n, d)       # Gaussian init (He, Xavier)
np.random.rand(n, d)        # Uniform [0, 1)

# --- RESHAPE & INDEXING ---
x.reshape(batch, seq, d)    # 3D tensor shape
x[np.newaxis, :]            # add batch dim
x.T                         # transpose (d, n) → (n, d)

# --- MATH ---
A @ B                       # matrix multiply
np.dot(q, k.T) / sqrt(d_k) # scaled attention scores
np.exp(x) / np.exp(x).sum()# softmax
np.maximum(0, x)            # ReLU
np.clip(grad, -1, 1)        # gradient clipping
np.linalg.norm(w)           # L2 norm (regularization)
np.argsort(scores)[::-1]    # rank by score (retrieval)

# --- STATS ---
x.mean(axis=0)              # feature means (batch norm)
x.std(axis=0)               # feature stds
(x - x.mean()) / x.std()    # standardize batch
```

Pandas Cheat Sheet

```
# --- INSPECTION ---
df.info()                   # shape, dtypes, memory
df.describe()               # descriptive stats
df.isnull().sum()           # missing value counts
df.value_counts('col')      # frequency of categories
df.corr()                   # correlation matrix

# --- CLEANING ---
df.dropna(subset=['label'])         # drop missing labels
df['f'].fillna(df['f'].median())    # impute median
df.drop_duplicates()                # remove duplicates
df.rename(columns={'old':'new'})    # rename columns

# --- FEATURE ENGINEERING ---
pd.get_dummies(df, columns=['cat'])           # one-hot encode
df['feat'].apply(np.log1p)                    # log transform
pd.cut(df['age'], bins=5)                     # binning
df.groupby('class')['feat'].transform('mean') # group stats

# --- SPLITTING FOR ML ---
```

```
train = df.sample(frac=0.8, random_state=42)
test = df.drop(train.index)
X = df.drop('target', axis=1).to_numpy(dtype='float32')
y = df['target'].to_numpy()
```